

2.1 Cart-Pole Swing-Up with Limited Actuation

2.1(a)

At SCP iteration i , linearize f_d around $(s_k^{(i)}, u_k^{(i)})$:

$$s_{k+1} = A_k s_k + B_k u_k + c_k,$$

where

$$A_k = \frac{\partial f_d}{\partial s}(s_k^{(i)}, u_k^{(i)}), \quad B_k = \frac{\partial f_d}{\partial u}(s_k^{(i)}, u_k^{(i)}),$$

$$c_k = f_d(s_k^{(i)}, u_k^{(i)}) - A_k s_k^{(i)} - B_k u_k^{(i)}.$$

The convex subproblem is

$$\begin{aligned} \min_{s, u} \quad & \sum_{k=0}^{N-1} ((s_k - s_{\text{goal}})^T Q (s_k - s_{\text{goal}}) + u_k^T R u_k) + (s_N - s_{\text{goal}})^T P (s_N - s_{\text{goal}}) \\ \text{s.t.} \quad & s_0 = \bar{s}_0, \\ & s_{k+1} = A_k s_k + B_k u_k + c_k, \\ & -u_{\max} \leq u_k \leq u_{\max}, \\ & \|s_k - s_k^{(i)}\|_{\infty} \leq \rho, \quad \|u_k - u_k^{(i)}\|_{\infty} \leq \rho. \end{aligned}$$

2.1(b)

The requested JAX implementation is:

```
1 A, B = jax.jacobian(fd, argnums=(0, 1))(sbar, ubar)
2 c = fd(sbar, ubar) - A @ sbar - B @ ubar
```

2.1(c)

The implementation is in `code/cartpole_swingup_constrained.py`. The function `scp_iteration` creates CVXPY variables for s and u , adds the linearized dynamics, box input constraints, trust region constraints, and minimizes the convex quadratic cost above.

2.1(d)

To swing up and then keep the pendulum upright, first use SCP to compute a feasible swing-up trajectory. Once the pendulum is near $(0, \pi, 0, 0)$, switch to a local LQR stabilizing controller designed around the upright equilibrium. This gives a hybrid controller: SCP for the large nonlinear maneuver, then LQR for local stabilization.

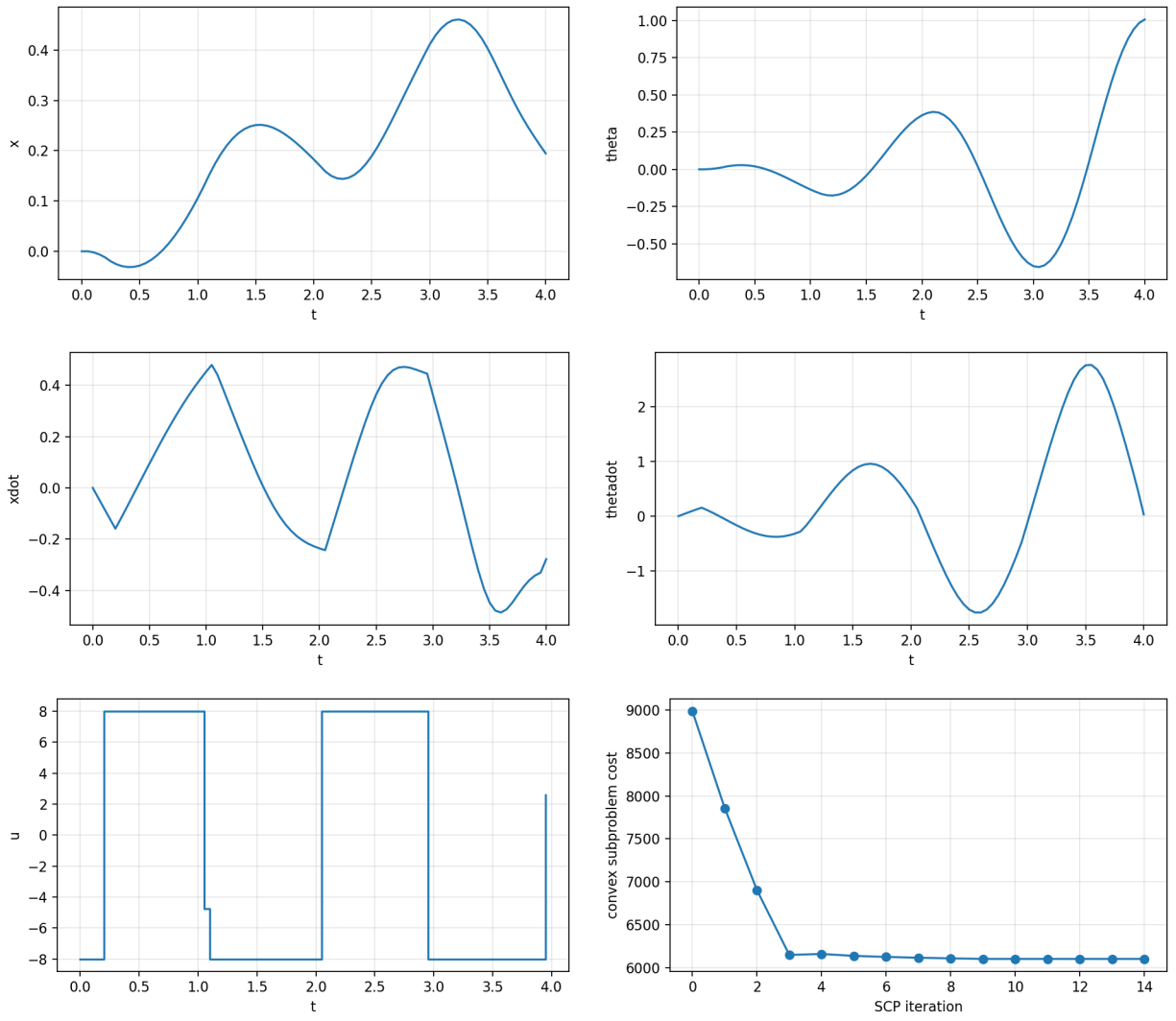


Figure 1: SCP swing-up trajectory with bounded input and trust-region updates.

2.2 Shortest Path Through a Grid

2.2(a)

Using dynamic programming from right to left gives the value function

node	$V(\text{node})$
B	0
u_5	10
d_5	11
u_4	18
m_4	16
d_4	20
u_3	25
um_3	23
dm_3	21
d_3	27
u_2	29
m_2	29
d_2	31
u_1	35
d_1	36
A	40

Thus the shortest path has cost

$$\boxed{40}.$$

One optimal path is

$$A \rightarrow u_1 \rightarrow u_2 \rightarrow um_3 \rightarrow u_4 \rightarrow u_5 \rightarrow B.$$

2.2(b)

For an n -segment grid, each route consists of n up moves and n down moves, so exhaustive search evaluates

$$\boxed{\binom{2n}{n}}$$

routes. Dynamic programming evaluates each nonterminal node. The number of such DP evaluations is

$$\boxed{n(n+2)}.$$

For $n = 3$, these give $\binom{6}{3} = 20$ and $3(5) = 15$, matching the problem statement.

2.3 Cart-Pole Balance

2.3(a)

With $\tilde{s}_k = s_k - \bar{s}$, Euler discretization gives

$$\tilde{s}_{k+1} \approx A\tilde{s}_k + Bu_k,$$

where

$$A = I + \Delta t \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{m_p g}{m_c} & 0 & 0 \\ 0 & \frac{(m_c + m_p)g}{m_c \ell} & 0 & 0 \end{bmatrix}, \quad B = \Delta t \begin{bmatrix} 0 \\ 0 \\ \frac{1}{m_c} \\ \frac{1}{m_c \ell} \end{bmatrix}.$$

2.3(b)

Using Riccati recursion until $\|P_k - P_{k-1}\|_{\max} < 10^{-4}$ gives

$$K_{\infty} = \begin{bmatrix} 0.73 & -231.85 & 4.22 & -68.25 \end{bmatrix}.$$

2.3(c)

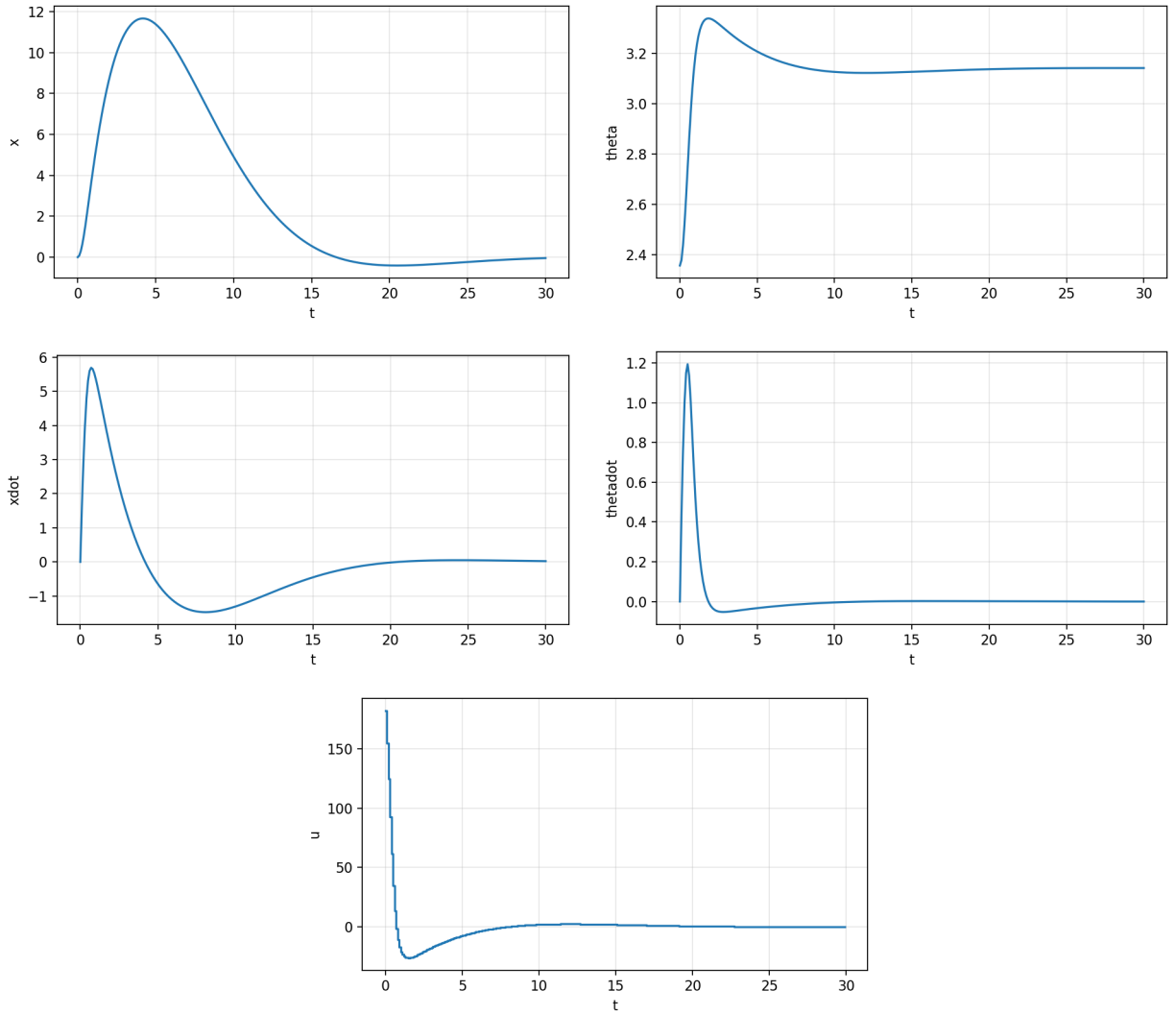


Figure 2: Cart-pole balance with the infinite-horizon LQR controller.

2.3(d)

- i. We can reuse A and B because the dynamics Jacobians along the reference do not depend on x , and the desired trajectory keeps $\theta = \pi$, $\dot{\theta} = 0$. Therefore the same upright linearization is valid along the reference.
- ii.

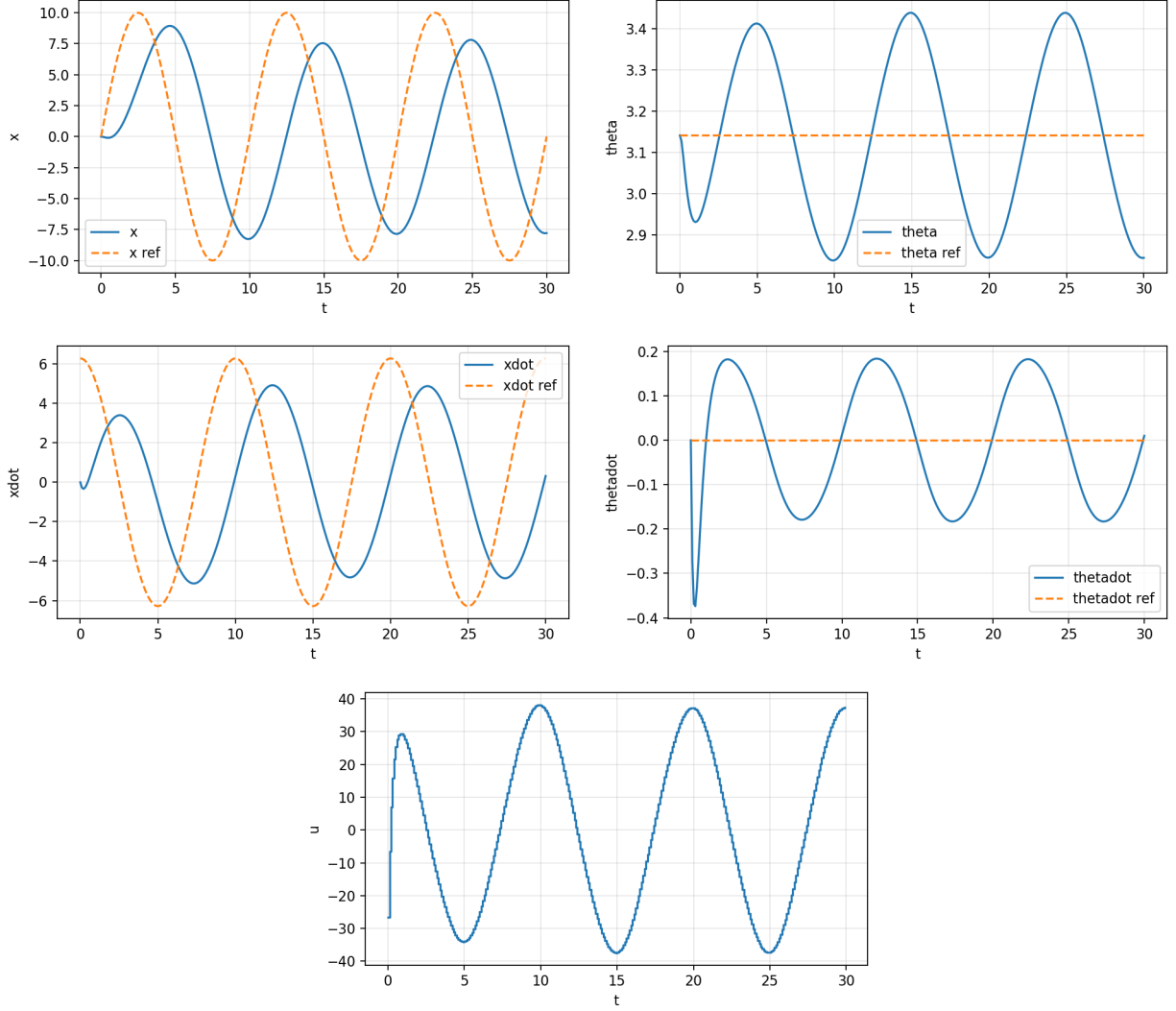


Figure 3: Tracking $x_{\text{ref}}(t) = 10 \sin(2\pi t/10)$ with LQR feedback.

- iii. The reference cart motion has nonzero acceleration, but the reference does not include a dynamically consistent feedforward force. The pendulum must tilt to generate the horizontal cart acceleration, so keeping $\theta(t) \equiv \pi$ while moving the cart sinusoidally is not a dynamically consistent upright trajectory without feedforward compensation.

2.4 Cart-Pole Swing-Up

2.4(a)

The requested one-line JAX linearization is

```
1 | A, B = jax.jacobian(fd, argnums=(0, 1))(sbar_k, ubar_k)
```

2.4(b)

Let

$$\tilde{s}_k = s_k - \bar{s}_k, \quad \tilde{u}_k = u_k - \bar{u}_k.$$

Expanding the quadratic cost gives linear terms

$$q_N = Q_N(\bar{s}_N - s_{\text{goal}}), \quad q_k = Q(\bar{s}_k - s_{\text{goal}}), \quad r_k = R\bar{u}_k.$$

2.4(c)

The iLQR implementation is in `code/cartpole_swingup.py`. It computes linearizations, performs the Riccati backward pass for feedback gains Y_k and offsets y_k , updates the nominal trajectory, and uses backtracking to keep the update stable.

2.4(d)

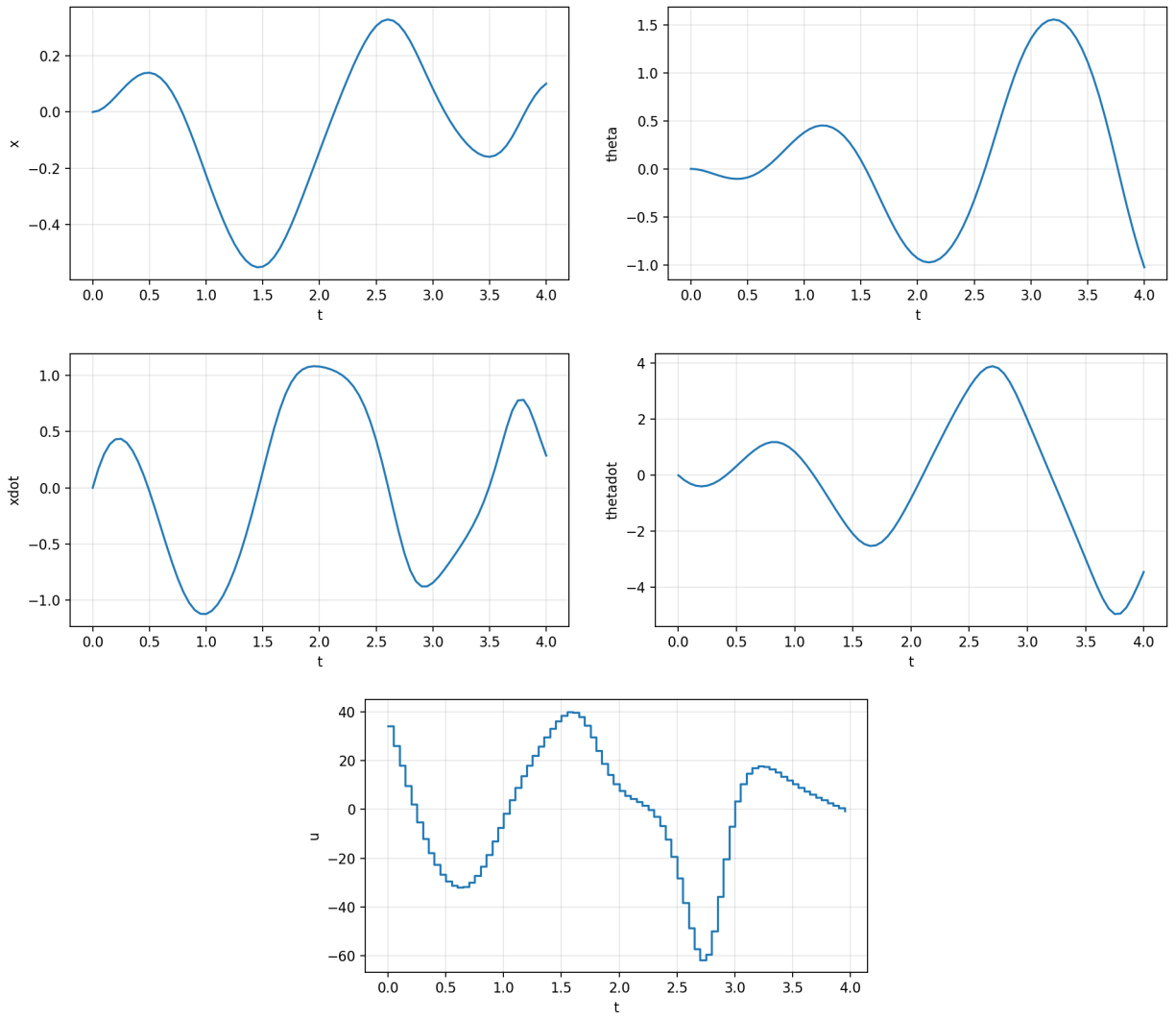


Figure 4: Open-loop application of the iLQR control sequence.

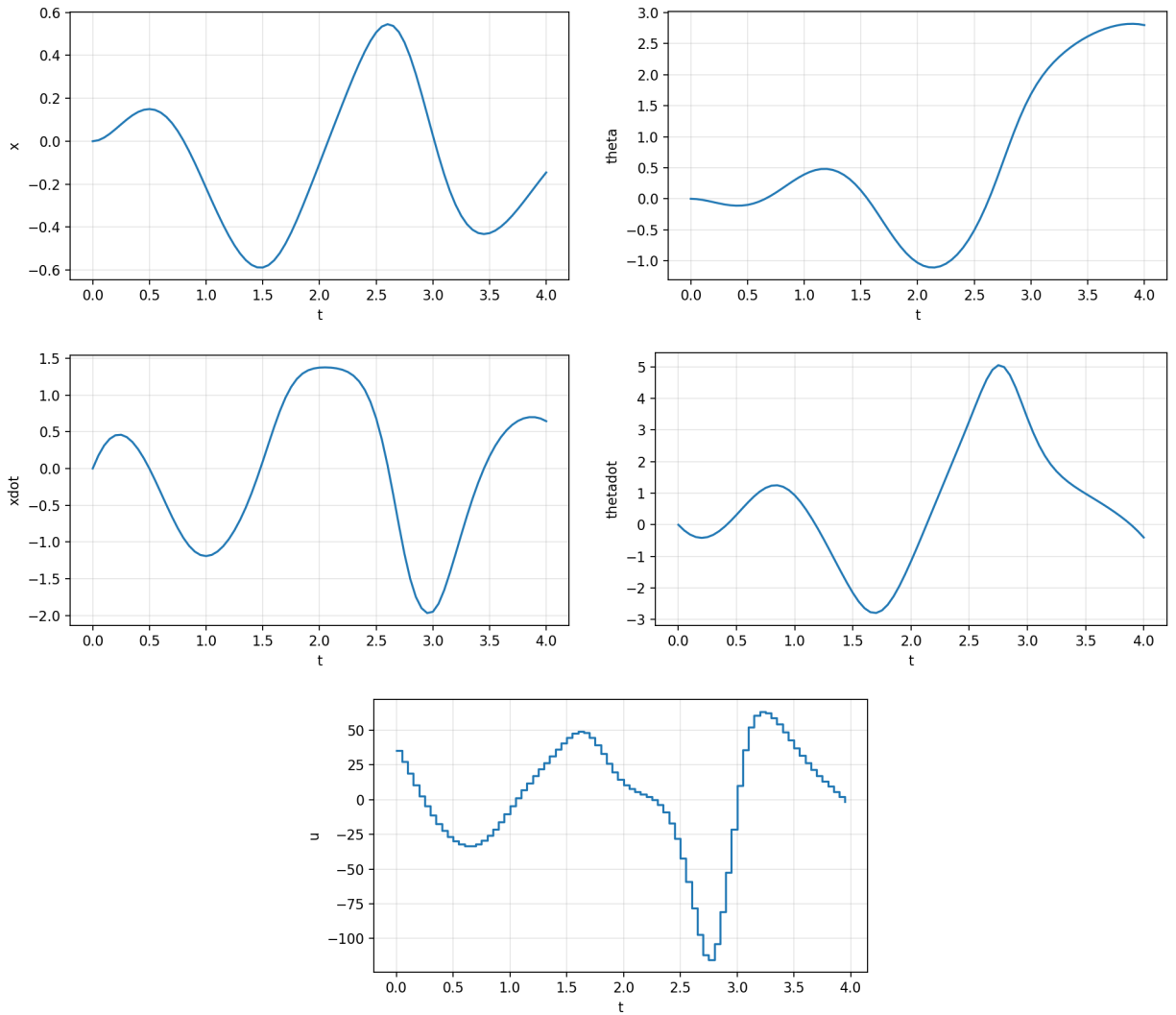


Figure 5: Closed-loop iLQR policy.

2.5 Machine Maintenance

Let $V_R(h)$ and $V_B(h)$ denote the optimal expected profit with h weeks remaining when the machine starts the week running or broken.

$$\begin{aligned}
 V_R(h) &= \max\{40 + 0.6V_R(h-1) + 0.4V_B(h-1), \\
 &\quad 30 + 0.3V_R(h-1) + 0.7V_B(h-1)\}, \\
 V_B(h) &= \max\{20 + 0.6V_R(h-1) + 0.4V_B(h-1), -50 + V_R(h-1)\}.
 \end{aligned}$$

With $V_R(0) = V_B(0) = 0$, the computed values are

h	0	1	2	3
$V_R(h)$	0	40	72	104
$V_B(h)$	0	20	52	84

The optimal policy is to perform preventive maintenance when running and to repair when broken. Since the machine is new at week 1 and guaranteed to run through that week, the total expected profit over four weeks is

$$100 + V_R(3) = 204.$$